



TRIBHUVAN UNIVERSITY  
INSTITUTE OF ENGINEERING  
PULCHOWK CAMPUS

PROJECT REPORT ON  
SPEAKER DIARIZATION IN NEPALI LANGUAGE

**SUBMITTED BY:**

SAMIP NEUPANE (PUL077BCT073)  
SANDESH POKHREL (PUL077BCT074)  
SANDESH PYAKUREL (PUL077BCT075)

**SUBMITTED TO:**

DEPARTMENT OF ELECTRONICS & COMPUTER ENGINEERING

April, 2025

# Page of Approval

TRIBHUVAN UNIVERSITY  
INSTITUTE OF ENGINEERING  
PULCHOWK CAMPUS  
DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

The undersigned certifies that they have read and recommended to the Institute of Engineering for acceptance of a project report entitled ”**SPEAKER DIARIZATION IN NEPALI LANGUAGE**” submitted by **Samip Neupane, Sandesh Pokhrel, Sandesh Pyakurel** in partial fulfillment of the requirements for the Bachelor’s degree in Electronics & Computer Engineering.

.....

Supervisor

**Dr. Basanta Joshi**

Department of Electronics and Computer  
Engineering,  
Pulchowk Campus, IOE, TU.

.....

External Examiner

**Sanjaya Subedi**

Co-Owner/Co-Founder  
TechAxis

Date of approval:

# Copyright

The author has agreed that the Library, Department of Electronics and Computer Engineering, Pulchowk Campus, and Institute of Engineering may make this report freely available for inspection. Moreover, the author has agreed that permission for extensive copying of this project report for scholarly purposes may be granted by the supervisors who supervised the work recorded herein or, in their absence, by the Head of the Department wherein the project report was done. It is understood that recognition will be given to the author of this report and the Department of Electronics and Computer Engineering, Pulchowk Campus, Institute of Engineering for any use of the material of this project report. Copying publication or the other use of this report for financial gain without the approval of to the Department of Electronics and Computer Engineering, Pulchowk Campus, Institute of Engineering, and the author's written permission is prohibited.

Request for permission to copy or to make any other use of the material in this report in whole or in part should be addressed to:

Head  
Department of Electronics and Computer Engineering  
Pulchowk Campus, Institute of Engineering, TU  
Lalitpur, Nepal.

# Acknowledgments

First and foremost, we wish to extend our heartfelt gratitude to Dr. Basanta Joshi, our project supervisor, whose generous sharing of knowledge, invaluable feedback, and constant encouragement have significantly contributed to the refinement and successful progress of our project. His expert guidance has been pivotal in shaping the direction of our research.

We are sincerely thankful to the Department of Electronics and Computer Engineering, Pulchowk Campus, for providing us with the platform, resources, and an environment conducive to the successful execution of our major project. The continued support and encouragement from the department have been instrumental throughout this journey.

We would also like to express our appreciation to our colleagues, friends, and peers for their enriching interactions, critical feedback, and unwavering support. Their intellectually stimulating discussions and collaborative spirit have greatly elevated the quality and depth of our work.

Any suggestions or constructive feedback regarding our work will be deeply valued and gratefully acknowledged.

# Abstract

Speaker diarization, the task of determining “who spoke when” in an audio recording, is a critical preprocessing step for a wide range of speech applications, from automated meeting transcription and call-center analytics to accessibility tools for the hearing impaired. However, most state-of-the-art diarization systems are optimized for high-resource languages (e.g., English) and struggle when applied to underrepresented, phonetically rich languages such as Nepali. To address this gap, this project develops a tailored speaker diarization framework for Nepali audio content.

The proposed system constructs a multilingual training corpus by aggregating and annotating speech from LibriSpeech, VoxCeleb, and dedicated Nepali and Hindi datasets. Preprocessing involves voice activity detection (VAD) to eliminate non-speech segments and data augmentation techniques to synthesize realistic multi-speaker scenarios. We explore End-to-End Neural Diarization (EEND) architectures including encoder–decoder attractor networks and Perceiver-based variants (DiaPer). Mixed-data training and explicit silent-segment modeling further enhances accuracy in application scenarios.

On Nepali and Hindi test sets containing up to four unknown speakers per recording, the *DiaPer* model achieved a Diarization Error Rate (DER) of 4.76%, while the encoder–decoder EEND model yielded a DER of 11.19%.

To facilitate real-world adoption, we implement a web application featuring a user-friendly React frontend and a Django backend. Users can upload arbitrary audio files, visualize speaker turns through an interactive timeline, and retrieve the corresponding transcribed text.

Keywords: *Speaker Diarization, End-to-End Neural Diarization (EEND), Voice Activity Detection (VAD), Nepali Language*

# Contents

|                                                      |          |
|------------------------------------------------------|----------|
| Page of Approval                                     | ii       |
| Copyright                                            | iii      |
| Acknowledgements                                     | iv       |
| Abstract                                             | v        |
| Contents                                             | viii     |
| List of Figures                                      | x        |
| List of Abbreviations                                | xi       |
| <b>1 Introduction</b>                                | <b>1</b> |
| 1.1 Background . . . . .                             | 1        |
| 1.2 Problem Statements . . . . .                     | 2        |
| 1.3 Objectives . . . . .                             | 2        |
| 1.4 Scope . . . . .                                  | 2        |
| <b>2 Literature Review</b>                           | <b>3</b> |
| 2.1 Related Work . . . . .                           | 3        |
| 2.2 Related Theory . . . . .                         | 4        |
| 2.2.1 Speaker Diarization Fundamentals . . . . .     | 4        |
| 2.2.2 Deep Learning in Speaker Diarization . . . . . | 4        |
| 2.2.3 End-to-End Neural Diarization (EEND) . . . . . | 5        |
| 2.2.4 Self-Attention Mechanisms . . . . .            | 5        |
| 2.2.5 Evaluation Metrics for Diarization . . . . .   | 5        |
| <b>3 Methodology</b>                                 | <b>6</b> |
| 3.1 Data Collection . . . . .                        | 6        |
| 3.1.1 Source Breakdown . . . . .                     | 6        |
| 3.2 Data Preprocessing . . . . .                     | 7        |
| 3.2.1 Voice Activity Detection (VAD) . . . . .       | 7        |

|          |                                                                          |           |
|----------|--------------------------------------------------------------------------|-----------|
| 3.2.2    | Audio Merging . . . . .                                                  | 7         |
| 3.2.3    | Audio Metadata and Merging Details . . . . .                             | 8         |
| 3.3      | Feature Extraction: Mel-Frequency Cepstral Coefficients (MFCC) . . . . . | 12        |
| 3.4      | Normalization . . . . .                                                  | 13        |
| 3.5      | Model Training . . . . .                                                 | 13        |
| 3.6      | Web Development . . . . .                                                | 14        |
| <b>4</b> | <b>Experimental Setup</b>                                                | <b>15</b> |
| 4.1      | Encoder-Decoder Based Diarization, EEND-EDA: . . . . .                   | 15        |
| 4.1.1    | Components . . . . .                                                     | 16        |
| 4.2      | EEND with Perceiver-Based Attractors . . . . .                           | 16        |
| 4.3      | Computational Costs . . . . .                                            | 17        |
| 4.3.1    | Training with EEND-EDA . . . . .                                         | 17        |
| 4.3.2    | Training with EEND-DiaPer . . . . .                                      | 17        |
| 4.4      | Diarization Error Rate (DER) . . . . .                                   | 18        |
| 4.4.1    | Different Models for DER . . . . .                                       | 18        |
| <b>5</b> | <b>System design</b>                                                     | <b>19</b> |
| 5.1      | EEND Model Pipeline Components . . . . .                                 | 19        |
| 5.1.1    | Frontend (Audio Input) . . . . .                                         | 19        |
| 5.1.2    | Feature Extraction . . . . .                                             | 19        |
| 5.1.3    | Encoder . . . . .                                                        | 19        |
| 5.1.4    | Attractor Calculation . . . . .                                          | 20        |
| 5.1.5    | Decoder & Speaker Labeling . . . . .                                     | 20        |
| 5.1.6    | RTTM Output . . . . .                                                    | 20        |
| 5.1.7    | Speech-to-Text Integration . . . . .                                     | 20        |
| <b>6</b> | <b>Results and Output</b>                                                | <b>21</b> |
| 6.1      | Training and Hyperparameter Configuration . . . . .                      | 21        |
| 6.2      | Dataset Composition . . . . .                                            | 21        |
| 6.2.1    | Training and Validation Curves . . . . .                                 | 22        |
| 6.3      | Testing & Output . . . . .                                               | 30        |
| 6.3.1    | Testing Without Noise Cancellation . . . . .                             | 30        |
| 6.3.2    | Testing With Noise Cancellation . . . . .                                | 30        |
| 6.4      | Testing Datasets and DER Evaluation . . . . .                            | 31        |
| 6.4.1    | Testing Datasets . . . . .                                               | 31        |
| 6.4.2    | DER Evaluation . . . . .                                                 | 32        |

|          |                                                         |           |
|----------|---------------------------------------------------------|-----------|
| 6.5      | Web Interface . . . . .                                 | 33        |
| <b>7</b> | <b>Limitations</b>                                      | <b>34</b> |
| 7.1      | Speaker Independent Accuracy and Robustness . . . . .   | 34        |
| 7.1.1    | Causes of Accuracy Drop . . . . .                       | 34        |
| 7.2      | Training Data Limitations . . . . .                     | 35        |
| 7.3      | Speaker Labeling Limitations . . . . .                  | 35        |
| <b>8</b> | <b>Future Enhancements</b>                              | <b>36</b> |
| 8.1      | LLM-Powered Speaker Labeling . . . . .                  | 36        |
| 8.2      | Visualization with Animations and Nepali Text . . . . . | 36        |
| 8.3      | Real-Time Speaker Diarization . . . . .                 | 37        |
|          | References . . . . .                                    | 38        |



# List of Figures

|      |                                                 |    |
|------|-------------------------------------------------|----|
| 1.1  | .....                                           | 1  |
| 3.1  | Methodology .....                               | 6  |
| 3.2  | Mel Frequency Cepstral Coefficients(MFCC) ..... | 13 |
| 4.1  | Encoder-Decoder Based EEND Architecture .....   | 15 |
| 4.2  | EEND-DiaPer Architecture .....                  | 17 |
| 5.1  | Encoder-Decoder Based EEND Architecture .....   | 19 |
| 6.1  | Train DER- 2spk .....                           | 22 |
| 6.2  | Valid DER - 2spk .....                          | 22 |
| 6.3  | Train activation loss - 2spk .....              | 22 |
| 6.4  | Valid activation loss - 2spk .....              | 22 |
| 6.5  | Train DER- 3spk .....                           | 23 |
| 6.6  | Valid DER - 3spk .....                          | 23 |
| 6.7  | Train activation loss - 3spk .....              | 23 |
| 6.8  | Valid activation loss - 3spk .....              | 23 |
| 6.9  | Train DER- 4spk .....                           | 24 |
| 6.10 | Valid DER - 4spk .....                          | 24 |
| 6.11 | Train activation loss - 4spk .....              | 24 |
| 6.12 | Valid activation loss - 4spk .....              | 24 |
| 6.13 | Train DER- Mspk .....                           | 25 |
| 6.14 | Valid DER - Mspk .....                          | 25 |
| 6.15 | Train activation loss - Mspk .....              | 25 |
| 6.16 | Valid activation loss - Mspk .....              | 25 |
| 6.17 | Train DER FA 2spk .....                         | 26 |
| 6.18 | Valid DER FA 2spk .....                         | 26 |
| 6.19 | Train DER miss 2spk .....                       | 26 |
| 6.20 | Valid DER miss 2spk .....                       | 26 |
| 6.21 | Train loss 2spk .....                           | 26 |
| 6.22 | Valid loss 2spk .....                           | 26 |
| 6.23 | Train DER FA 3spk .....                         | 27 |

|      |                                                                |    |
|------|----------------------------------------------------------------|----|
| 6.24 | Valid DER FA 3spk . . . . .                                    | 27 |
| 6.25 | Train DER miss 3spk . . . . .                                  | 27 |
| 6.26 | Valid DER miss 3spk . . . . .                                  | 27 |
| 6.27 | Train loss 3spk . . . . .                                      | 27 |
| 6.28 | Valid loss 3spk . . . . .                                      | 27 |
| 6.29 | Train DER FA 4spk . . . . .                                    | 28 |
| 6.30 | Valid DER FA 4spk . . . . .                                    | 28 |
| 6.31 | Train DER miss 4spk . . . . .                                  | 28 |
| 6.32 | Valid DER miss 4spk . . . . .                                  | 28 |
| 6.33 | Train loss 4spk . . . . .                                      | 28 |
| 6.34 | Valid loss 4spk . . . . .                                      | 28 |
| 6.35 | Train DER FA Mspk . . . . .                                    | 29 |
| 6.36 | Valid DER FA Mspk . . . . .                                    | 29 |
| 6.37 | Train DER miss Mspk . . . . .                                  | 29 |
| 6.38 | Valid DER miss Mspk . . . . .                                  | 29 |
| 6.39 | Train loss Mspk . . . . .                                      | 29 |
| 6.40 | Valid loss Mspk . . . . .                                      | 29 |
| 6.41 | Incorrect diarization without noise cancellation. . . . .      | 30 |
| 6.42 | Correct diarization after applying noise cancellation. . . . . | 31 |
| 6.43 | Web App demonstration of Speaker Diarization. . . . .          | 33 |

# List of Abbreviations

|                        |                                                                |
|------------------------|----------------------------------------------------------------|
| <b>EEND</b>            | End-to-End Neural Diarization                                  |
| <b>EDA</b>             | Encoder-Decoder based Attractors                               |
| <b>DER</b>             | Diarization Error Rate                                         |
| <b>LSTM</b>            | Long Short-Term Memory Networks                                |
| <b>EEND-EDA</b>        | End-to-End Neural Diarization Encoder-Decoder based Attractors |
| <b>BLSTM</b>           | Bidirectional Long Short-Term Memory Networks                  |
| <b>CNN</b>             | Convolutional Neural Network                                   |
| <b>MFCC</b>            | Mel-frequency Cepstral Coefficient                             |
| <b>RNN</b>             | Recurrent Neural Network                                       |
| <b>EEND-PBA</b>        | End-to-End Neural Diarization-Perceiver-Based Attractors       |
| <b>RTTM</b>            | Rich Transcription Time Marked                                 |
| <b>SA</b>              | Self-Attention                                                 |
| <b>PIT</b>             | Permutation Invariant Training                                 |
| <b>2spk, 2-speaker</b> | Audio containing two unique speakers                           |
| <b>3spk, 3-speaker</b> | Audio containing three unique speakers                         |
| <b>4spk, 4-speaker</b> | Audio containing four unique speakers                          |
| <b>Mspk, M-speaker</b> | Variable number of unique speakers(upto 4) per audio           |

# 1. Introduction

## 1.1 Background

Speaker diarization is the task of partitioning an audio stream into homogeneous speaker segments - essentially answering “who spoke when” in multispeaker environments. It serves as a foundational preprocessing step for various speech applications, including automatic speech recognition, speaker indexing, and real-time captioning. This technique supports numerous services, from automated meeting transcription and call center analytics to accessibility tools for the hearing impaired.

Traditional diarization approaches typically cluster speaker embeddings but face limitations handling overlapping speech and directly minimizing diarization errors. The advent of deep learning has shifted the field toward end-to-end neural methodologies that model speaker activities jointly as a multi-label classification problem. End-to-End Neural Diarization (EEND) employs self-attention mechanisms to capture global context and effectively manage speaker overlap. Further enhancements, such as permutation-free objective functions, simplify model pipelines and directly optimize diarization performance.

Despite these advancements in high-resource languages, speaker diarization research for many low-resource and underrepresented languages remains sparse. Nepali, a phonetically rich Indo-Aryan language, poses distinct challenges for speaker diarization in real-world contexts. Hence, in this project we aim to take advantage of these end-to-end neural techniques and related advances to develop a speaker diarization system tailored for Nepali language that achieves improved accuracy and robustness.

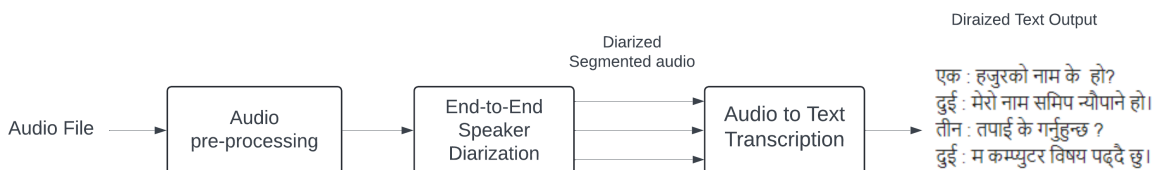


Figure 1.1:

## 1.2 Problem Statements

Existing speaker diarization systems are almost exclusively developed and optimized for high-resource languages (e.g., English), using large-scale annotated corpora and extensive pre-training. When applied to Nepali audio, these systems often underperform due to Nepali’s distinct phonetic and prosodic characteristics, the scarcity of time-aligned speaker labels across conversational, broadcast, and meeting contexts. There is therefore a critical need for a diarization framework that is specifically tailored to Nepali language capable of accurately partitioning audio into homogeneous speaker segments to support various applications.

## 1.3 Objectives

The objectives of Speaker Diarization in Nepali Language project are as follows:

- Utilize End-to-End Neural Diarization (EEND) methods for Nepali language.
- Implement a web interface to allow users to upload audio files and display transcription and visualization.

## 1.4 Scope

The scope of this project involves the collection and annotation of diverse audio data including Nepali, Hindi languages, the development and fine-tuning of advanced diarization models such as EEND-EDA and EEND with Perceiver-Based Attractors, and extensive performance evaluation using standard metrics like Diarization Error Rate (DER). The project also includes the deployment of user-friendly interfaces and tools to integrate the diarization system into practical applications.

## 2. Literature Review

### 2.1 Related Work

Speaker diarization—automatically segmenting audio into regions according to who is speaking—has traditionally relied on clustering-based pipelines (e.g., Gaussian Mixture Models or x-vectors with clustering). While effective in many settings, these methods are not trained end-to-end to minimize diarization error rate (DER) and often struggle when speakers overlap.

Fujita and Nakamura [1] address these limitations in their paper *End-to-End Neural Speaker Diarization with Self-Attention*. They replace the bidirectional LSTM blocks in the original EEND framework with self-attention layers that attend to all time frames simultaneously. This global context modeling enables the network to more accurately distinguish overlapping speech segments. Empirical results on standard benchmarks show a substantial reduction in DER, particularly in two-speaker overlap scenarios.

Horiguchi *et al.* [2] propose *End-to-End Speaker Diarization for an Unknown Number of Speakers with Encoder-Decoder Based Attractors*. Their method uses an encoder-decoder architecture to generate a variable number of “attractors” from a self-attentive embedding sequence. Each attractor represents one speaker and interacts with the same embeddings to yield per-speaker activity probabilities. This dynamic attractor mechanism adapts seamlessly to conversations with unknown and varying speaker counts, though with additional computational overhead during attractor generation.

Landini *et al.* [3] introduce *DiaPer: End-to-End Neural Diarization with Perceiver-Based Attractors*, replacing the encoder-decoder attractor module with a Perceiver-based attractor network. By leveraging the Perceiver’s cross-attention to project variable-length inputs into a fixed set of latent vectors, DiaPer achieves better DER on the CallHome dataset, more accurate speaker-count estimation, and faster inference compared to EEND-EDA. Their lightweight design attains state-of-the-art performance across over ten public wide-band datasets, and they release both code and pretrained models for reproducibility.

Comparing these approaches, self-attention EEND excels at leveraging long-range dependencies to separate overlapping speakers but assumes a fixed speaker count during training. EEND-EDA adds flexibility for variable speaker numbers through its attractor decoder, though this iterative synthesis increases runtime cost. DiaPer unifies the strengths of both: the Perceiver latent encoder provides global context aggregation with linear complexity,

while its attractor module dynamically adapts to speaker count in a single non-iterative pass. Together, these end-to-end methods mark significant progress in diarization accuracy, efficiency, and flexibility, pointing toward future models that might integrate Perceiver latent encoding, self-attention context modeling, and dynamic attractor synthesis into a single cohesive framework.

## **2.2 Related Theory**

### **2.2.1 Speaker Diarization Fundamentals**

Speaker diarization addresses the fundamental question of "who spoke when?" by segmenting audio streams according to speaker identity. The field has undergone substantial transformation since its early clustering-based approaches, which merely grouped acoustically similar segments without explicitly minimizing diarization errors or adequately handling speaker overlap. Traditional pipeline architectures consisting of voice activity detection, speaker segmentation, feature extraction, and clustering struggle particularly in multi-speaker environments where acoustic interference and conversational dynamics create complex patterns of overlapping speech. These limitations are especially problematic in critical applications such as meeting transcription services, courtroom proceedings, broadcast news analysis, and contact center quality assurance, where speaker attribution directly impacts downstream analysis and decision-making processes.

### **2.2.2 Deep Learning in Speaker Diarization**

The emergence of deep learning techniques has fundamentally reshaped speaker diarization by eliminating the need for hand-crafted feature engineering and discrete clustering stages in favor of trainable, end-to-end neural architectures. Initial deep approaches leveraged convolutional neural networks (CNNs) to extract discriminative spectro-temporal representations directly from raw or minimally processed audio inputs. Researchers subsequently integrated recurrent architectures—particularly bidirectional long short-term memory (BLSTM) networks—to model the temporal evolution of speaker characteristics across extended utterances. The most recent advances incorporate self-attention mechanisms and Transformer-derived architectures, which have demonstrated superior capability in capturing global contextual dependencies and modeling complex overlap behaviors with computational efficiency that far exceeds traditional recurrent models. These integrated neural frameworks jointly optimize the extraction of acoustic features, speaker embedding generation, and temporal segmentation to directly minimize diarization error rate (DER), resulting in systems that exhibit remarkable robustness to acoustic variability, channel effects, and speaker diversity.

### 2.2.3 End-to-End Neural Diarization (EEND)

End-to-End Neural Diarization (EEND) represents a paradigm shift in diarization methodology, transcending the inherent limitations of cascaded, clustering-dependent approaches. EEND’s distinctive architecture replaces conventional bidirectional LSTM networks with self-attention mechanisms to simultaneously perform speaker identification and temporal segmentation directly from acoustic input. This integrated approach yields several critical advantages: It eliminates error propagation between system components, directly optimizes for diarization accuracy rather than proxy objectives, and demonstrates exceptional proficiency in handling overlapping speech, a notoriously challenging aspect of real-world conversational dynamics. The self-attention framework enables EEND to capture intricate speaker-specific patterns by attending to both adjacent and distant frames, facilitating the discrimination of speaker characteristics even in acoustically challenging environments with multiple active speakers.

### 2.2.4 Self-Attention Mechanisms

Self-attention mechanisms have revolutionized sequence modeling tasks by enabling direct computation of relationships between arbitrary elements in an input sequence, regardless of their positional distance. Unlike recurrent architectures that process information sequentially through time, self-attention computes attention weights that quantify the relevance of every frame to every other frame within the sequence. This parallel processing capability provides two crucial benefits for speaker diarization: first, it enables the model to capture both local acoustic patterns and global conversational structure; second, it allows for more efficient training and inference compared to recurrent approaches. When integrated into the EEND framework, self-attention blocks create powerful representations that encode speaker-specific characteristics while maintaining temporal coherence, thereby enhancing the system’s capacity to track multiple speakers in dynamic conversational settings.

### 2.2.5 Evaluation Metrics for Diarization

The assessment of speaker diarization systems relies on a suite of complementary metrics, with Diarization Error Rate (DER) serving as the primary performance indicator. DER quantifies the fraction of time incorrectly attributed to speakers through a weighted combination of missed speech detection, false alarm instances, and speaker confusion errors. These metrics, provide crucial feedback for model optimization and enable meaningful comparison across different diarization approaches.



## 3. Methodology

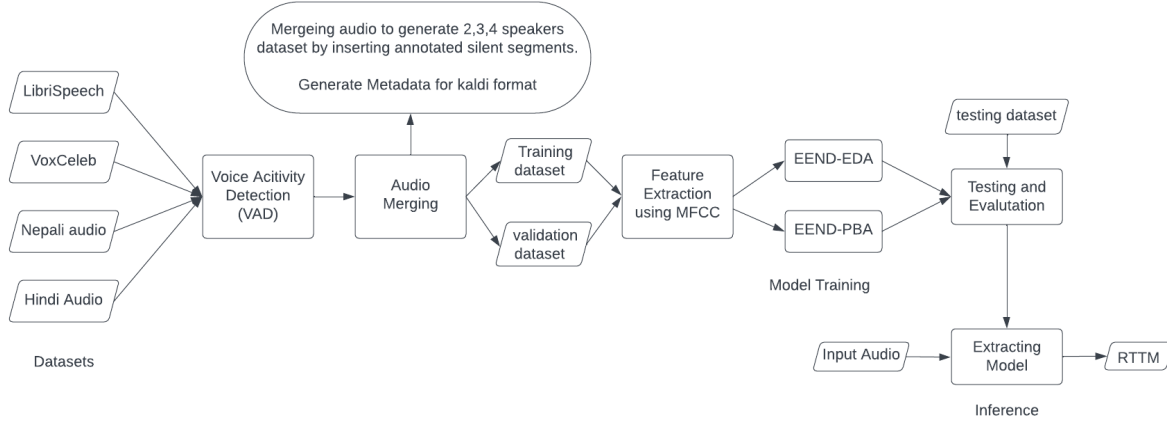


Figure 3.1: Methodology

### 3.1 Data Collection

We compiled a multilingual and multi-context audio dataset by aggregating speech samples from several established public datasets. The selected datasets were chosen to capture a broad spectrum of speaker characteristics—such as gender, accent, and language—and to include both clean studio-quality recordings and real-world conversational data. This diverse mix enables our system to perform effectively across varied acoustic conditions and speaker profiles.

#### 3.1.1 Source Breakdown

The following datasets were collected:

- **LibriSpeech:** A widely used corpus featuring clean, high-quality speech from 921 speakers. Its consistent recording conditions and clear audio quality make it ideal for building better speech models [4].
- **VoxCeleb:** This dataset includes recordings from 1211 speakers, offering a more diverse set of voices. The broad range of speaker characteristics—such as age, gender, and accent—helps improve our system’s generalization to real-world variability [5].

- **Nepali Female Speakers:** Collected by Google in Nepal, this dataset comprises audio from 18 native Nepali female speakers. It provides high-quality samples essential for incorporating language of interest into our model [6].
- **Hindi Audio:** Featuring contributions from 100 unique speakers, this dataset enriches our linguistic diversity and enhances performance across South Asian language contexts [7].

## 3.2 Data Preprocessing

Data preprocessing is an essential part of the pipeline for the End-to-End Diarization (EEND) model. It prepares the raw data, ensuring that the model only focuses on relevant speech and handles challenges like noise.

### 3.2.1 Voice Activity Detection (VAD)

To ensure the model does not confuse silence with speech, we used a Voice Activity Detection (VAD) technique, specifically employing **webrtcvad**, a Python library. This method identifies and removes non-speech segments of audio, thus eliminating the silent portions, which might otherwise be falsely recognized as speaker activity by the model. Removing silence ensures the model focuses only on segments where human speech is detected.

### 3.2.2 Audio Merging

One of the key preprocessing steps in our methodology involved generating synthetic multi-speaker audio by merging recordings from different individual speakers. This approach was necessary because some datasets—such as LibriSpeech, VoxCeleb, Nepali Female Speakers, and Hindi Audio—consist of isolated speech recordings, where each audio file contains only a single speaker. To simulate realistic multi-speaker scenarios, we created combined segments featuring 2, 3, and 4 speakers.

#### Merging Single-Speaker Recordings

We first split the single-speaker datasets into training, validation, and test sets in a 60:20:20 ratio. To generate multi-speaker audio:

- We randomly selected recordings from different speakers and merged them to create segments containing 2, 3, or 4 speakers.
- An additional mixed dataset was created that includes a combination of 2, 3, and 4 speakers within a single segment.

- During merging, we inserted short periods of silence between speaker turns to better mimic natural conversational patterns.
- Parameters such as minimum and maximum speech duration per speaker, silence insertion probability, and silent interval duration were manually tuned to ensure diversity and realism.

### Merged Dataset Summary

The final distribution of merged audio files used for training, and validation is shown in Table 3.1.

Table 3.1: Summary of Merged Multi-Speaker Audio Dataset

| Speaker Type | Data Source                    | Sample Count              |
|--------------|--------------------------------|---------------------------|
| 2 speakers   | Merged single-speaker datasets | Train: 10000<br>Val: 3000 |
| 3 speakers   | Merged single-speaker datasets | Train: 10000<br>Val: 3000 |
| 4 speakers   | Merged single-speaker datasets | Train: 10000<br>Val: 3000 |
| 2,3,4 mix    | Merged single-speaker datasets | Train: 10000<br>Val: 3000 |

### 3.2.3 Audio Metadata and Merging Details

We maintained detailed metadata records for the processed audio using the **Kaldi format**, which is a well-established format in the speech processing community. This included four important files:

- **wav.scp**: Lists all audio files used in the training.
- **utt2spk**: Maps each utterance (or audio segment) to a speaker.
- **reco2dur**: Contains the duration information for each audio file.
- **segments**: Defines the start and end times for the speech segments within each audio file.

## reco2dur File

The `reco2dur` file provides duration information for each audio recording in the dataset. Each line in the file follows the format:

```
<recording_id> <duration>
```

The contents of the `reco2dur` file are as follows:

```
103-1240-0000 14.085
103-1240-0001 15.945
103-1240-0002 13.945
```

The `reco2dur` file is crucial for:

- **Managing Audio Data:** Knowing the duration of each audio file helps in organizing and processing the data efficiently.
- **Speech Processing:** Accurate duration information is essential for tasks such as segmentation, alignment, and feature extraction in speech processing systems.
- **Training and Evaluation:** Ensures that the audio files are processed with their exact durations, which is vital for training and evaluating speech recognition models.

## ‘wav.scp’ File

The `wav.scp` file lists all the audio files used in the training or evaluation dataset, mapping each unique recording ID to its file path. This file is crucial for specifying where the audio data is stored. The format of each entry in the `wav.scp` file is:

```
<recording_id> <file_path>
```

The contents of the `wav.scp` file are as follows:

```
00000000 main/dataset/train_dataset/audio/00000000.flac
00000001 main/dataset/train_dataset/audio/00000001.flac
00000002 main/dataset/train_dataset/audio/00000002.flac
```

The `wav.scp` file serves the following purposes:

- **Audio File Mapping:** It maps each recording ID to its corresponding file path, allowing the system to access and process the audio files based on these identifiers.

- **Data Access:** Facilitates the retrieval of audio files during training or evaluation by providing a direct reference to their locations.
- **Integration with Other Files:** Works in conjunction with other files such as `utt2spk` and `segments` to ensure that the audio data is properly utilized in speech processing tasks.

### ‘utt2spk’ File

The `utt2spk` file maps each utterance (or audio segment) to a specific speaker. This mapping is essential for tasks such as speaker identification and speaker diarization. The format of each entry in the `utt2spk` file is:

```
<utterance_id> <speaker_id>
```

The contents of the `utt2spk` file are as follows:

```
103-1240-0000 103
103-1240-0001 103
103-1240-0002 103
103-1240-0003 103
103-1240-0004 103
103-1240-0005 103
103-1240-0006 103
103-1240-0007 103
```

The `utt2spk` file serves the following purposes:

- **Speaker Mapping:** It maps each utterance ID to a specific speaker ID, allowing the system to identify which speaker is associated with each audio segment.
- **Speaker Identification:** Facilitates tasks related to speaker identification and diarization by providing the necessary mapping between utterances and speakers.
- **Integration with Other Files:** Works in conjunction with files such as `wav.scp` and `segments` to ensure that audio data is accurately linked to speaker information for processing and analysis.

## ‘segments’ File

The `segments` file provides timing information for speech segments within each audio file. This file is crucial for tasks such as speech alignment and segmentation, where knowing the precise start and end times of each segment is essential. The format of each entry in the `segments` file is:

```
<recording_id> <utterance_id> <start_time> <end_time>
```

The contents of the `segments` file are as follows:

```
460-172359-0060 00000000 0.00 4.89
233-134440-0040 00000000 4.89 8.18
5163-18515-0005 00000000 8.18 17.86
```

The `segments` file serves the following purposes:

- **Segmentation:** It provides detailed timing information for each segment within the audio files, including the start and end times. This is essential for accurately segmenting audio for further processing.
- **Alignment:** Facilitates the alignment of audio data with transcripts or other annotations by providing precise time boundaries for each segment.
- **Integration with Other Files:** Works in conjunction with files such as `wav.scp` and `utt2spk` to ensure that segmentation information is properly linked to the audio data and speaker information for comprehensive analysis.

## ‘merge’ File

The `merge` file consolidates multiple recording IDs into a single entry, grouping together several recordings that are related or need to be processed together. The format of each entry in the `merge` file is:

```
<group_id> <recording_id_1> <recording_id_2> ... <recording_id_n>
```

The contents of the `merge` file are as follows:

00000000 460-172359-0060 233-134440-0040 5163-18515-0005 8465-246942-0011  
00000001 2514-149482-0009 2843-152918-0037 298-126790-0012 1069-133709-0031  
458-126290-0006  
00000002 5652-19215-0051 8312-279791-0008 3830-12530-0041  
00000003 8098-278252-0016 3240-131231-0057 201-122255-0026 8226-274371-0033  
730-360-0020  
00000004 2136-5147-0021 83-9960-0003 2911-15084-0000 6385-34655-0017  
32-4137-0046  
00000005 8747-293952-0104 6925-80680-0074

The `merge` file serves the following purposes:

- **Grouping Recordings:** It groups multiple recording IDs under a single group ID, which helps in managing related recordings or processing them together.
- **Efficient Processing:** Facilitates efficient processing or analysis of grouped recordings by consolidating them into a single reference.
- **Integration with Other Files:** Works in conjunction with files such as `wav.scp` and `utt2spk` to ensure that all relevant recordings are processed as a unit for tasks like training, evaluation, or feature extraction.

This structured metadata is essential for training and validating the EEND model, ensuring the model can distinguish between different speakers based on the correct segment assignments.

### 3.3 Feature Extraction: Mel-Frequency Cepstral Coefficients (MFCC)

Once the audio files were preprocessed, we moved to feature extraction using **Mel-Frequency Cepstral Coefficients (MFCC)**, a widely used technique in speech recognition. MFCCs represent the short-term power spectrum of an audio signal, capturing essential features related to human speech while discarding noise and irrelevant data. For the EEND model, these features help in distinguishing between different speakers based on their vocal characteristics.

MFCC feature extraction involves the following steps:

1. **Framing:** Breaking the audio into small frames.
2. **Windowing:** Applying a windowing function (like Hamming) to reduce spectral leakage.

3. **Fourier Transform:** Converting the windowed frames from the time domain to the frequency domain.
4. **Mel Scale:** Mapping the frequencies to the Mel scale, which mimics how humans perceive sound.
5. **Cepstrum Calculation:** Taking the logarithm of the Mel-scale power spectrum and applying the inverse Fourier Transform to obtain the MFCC coefficients.

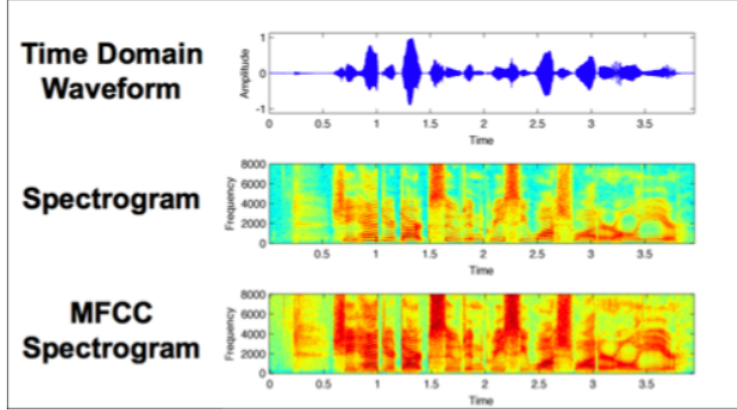


Figure 3.2: Mel Frequency Cepstral Coefficients(MFCC)

### 3.4 Normalization

Normalization is a crucial step in the context of the EEND model, ensuring consistency in the range of feature values across different audio files. The variation in volume levels, speech intensity, or even background noise across different audio files can affect the model’s ability to detect and differentiate speakers. **Normalization** helps in reducing this variance by adjusting the features to a common scale, thus ensuring that the differences captured by the model are primarily due to speaker characteristics and not due to recording conditions or noise.

By normalizing the MFCC features, we make the data more uniform and improve the performance of the machine learning algorithms used in EEND. This step is vital for improving accuracy in speaker recognition, especially in complex multi-speaker environments.

### 3.5 Model Training

To evaluate the effectiveness of our audio merging and preprocessing pipeline, we trained two state-of-the-art speaker diarization models: **EEND-EDA** (End-to-End Neural Diarization with Encoder-Decoder Attractors)[2] and **DiaPer** (End-to-End Neural Diarization with



Perceiver-Based Attractors)[3]. We obtained the code for EEND-EDA [8][9] and EEND-PBA from BUTSpeechFIT.

Both models were trained separately on four different configurations:

- **2-speaker** datasets
- **3-speaker** datasets
- **4-speaker** datasets
- **2,3,4 mixed-speaker** datasets

Each configuration used the respective training, validation, and test splits described earlier. For fair comparison and consistent benchmarking, the same data splits were used for both models across all configurations. Evaluation was performed using Diarization Error Rate (DER) as the primary metric.

The goal of this multi-configuration training was to assess how each model adapts to varying speaker counts and to evaluate their robustness in both fixed and mixed-speaker scenarios.

## 3.6 Web Development

To provide an accessible interface for users to interact with our diarization models, we developed a lightweight web application. The frontend was built using **React**, offering a responsive and intuitive user experience. The backend was implemented with **Django**, serving as the core engine to handle user requests, manage model inference, and deliver results efficiently.

The application allows users to upload audio files directly through the interface. Once uploaded, the backend processes the audio using the trained diarization models and returns the following:

- A transcription of the diarized audio
- Speaker-labeled segments with accurate timestamps
- Visualizations showing speaker turns and segment boundaries

The results are displayed in a clean and interactive format, helping users easily understand who spoke when and for how long. This tool serves as a demonstration of our system’s real-world applicability and provides a user-friendly way to test diarization on custom audio inputs.

## 4. Experimental Setup

Our project implements a diarization workflow that explores two distinct approaches within a unified framework. The first approach leverages an Encoder-Decoder architecture, following a sequence-to-sequence modeling paradigm, where the encoder processes input audio features into latent representations, and the decoder sequentially generates speaker attractors. This design allows the model to dynamically infer speaker identities and their speaking segments. The second approach introduces Perceiver-based attractors, utilizing the Perceiver’s cross-attention mechanism to efficiently aggregate frame-level representations into speaker-specific attractors. This method enhances scalability and generalization, particularly for handling variable-length audio and overlapping speech. Together, these approaches enable a comprehensive comparison of traditional seq2seq and modern attention-driven architectures in end-to-end speaker diarization. Finally, thus generated audio is then transformed into text using Google’s Speech-to-Text API.

### 4.1 Encoder-Decoder Based Diarization, EEND-EDA:

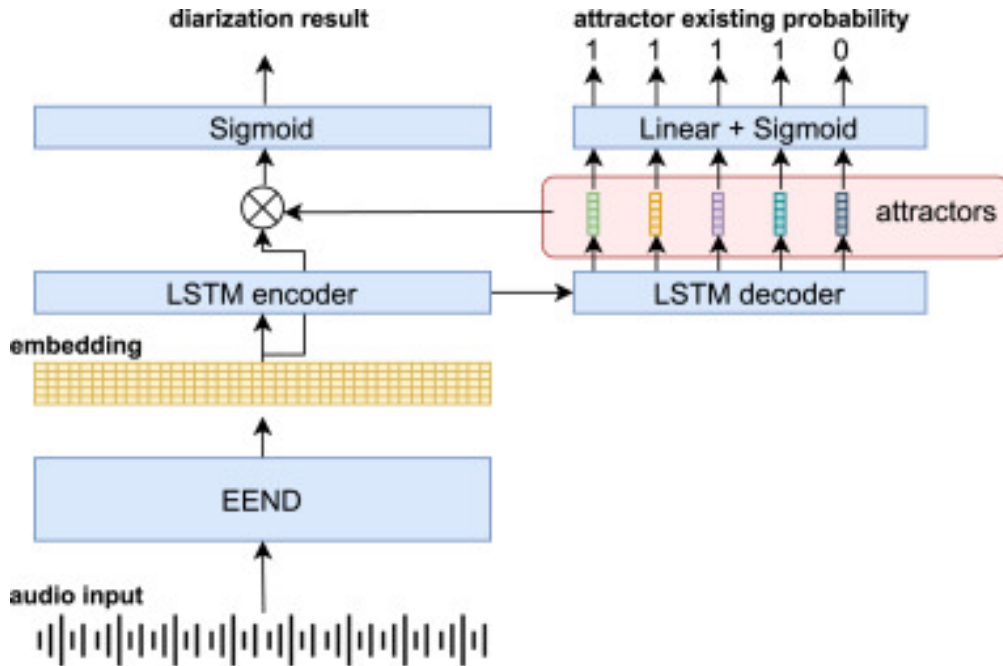


Figure 4.1: Encoder-Decoder Based EEND Architecture

### 4.1.1 Components

- **Encoder:**

**Input:** Raw audio waveforms or feature representations (e.g., spectrograms). It encodes the input into a latent space, producing a high-level representation that captures essential information while reducing dimensionality. It consists of LSTMs to capture both spatial and temporal features.

**Function:** Produces a set of attractors, each representing a distinct source (e.g., different speakers). Attractors are learned representations that serve as anchors or reference points in the latent space. They are used to pull the latent representations of similar data points closer to them. It is generated by learning through a neural network layer that outputs multiple attractors. It consists of multiple layers of LSTMs to capture the frames and provide the context vector for the LSTM decoder.

- **Decoder:**

**Input:** A sequence of embeddings which captures the acoustic and speaker-specific context at time  $t$ .

**Function:** Generates the attractors which identifies each frame and assigns it with the respective speaker identity. Thus generated attractors are again passed as the input to the softmax layer. It consists of LSTMs which takes the embeddings from the previous Embedding layer consisting of context vector generated by Encoder.

## 4.2 EEND with Perceiver-Based Attractors

The Perceiver is a type of Transformer model designed to handle high-dimensional data more efficiently. In the context of EEND-DiaPer, it works by first compressing raw audio frames into a smaller, fixed-size latent space. This step helps manage the computational complexity that comes with processing long audio sequences. Next, the model uses cross-attention mechanisms to iteratively refine these compressed representations. Unlike traditional RNNs or standard Transformers, which can struggle with scalability when dealing with large inputs, the Perceiver’s approach allows it to maintain performance without becoming overly resource-intensive. In EEND-DiaPer, this architecture replaces conventional decoder structures (like RNNs or basic Transformers) to better handle variable numbers of speakers and overlapping speech. By doing so, it improves both efficiency and adaptability, making the system more robust for real-world diarization tasks where speaker counts and audio lengths can vary widely.

Perceiver-Based EEND replaces the LSTM decoder which is used in the EEND-EDA with the use of latent vectors which acts as the query vector for the generation of the attractors.

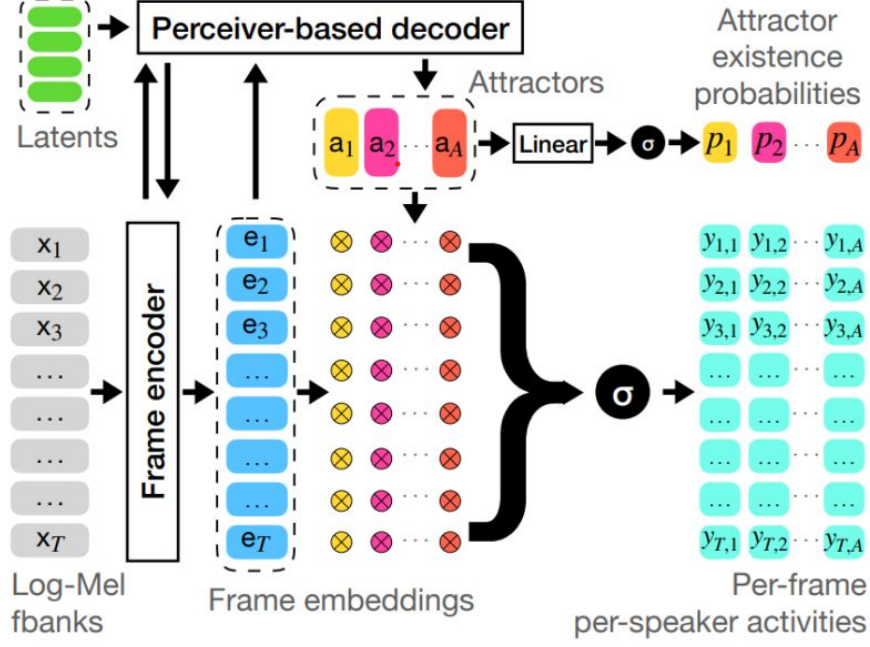


Figure 4.2: EEND-DiaPer Architecture

This decoder gets the latent vector along with the frame encoder as input. This architecture generates the  $T \times A$  matrix as final output.

## 4.3 Computational Costs

### 4.3.1 Training with EEND-EDA

Training the EEND-EDA model on a 196GB audio dataset (covering 2-speaker, 3-speaker, 4-speaker, and mixed scenarios) in 4 shifts with an average of 40 epochs required significant computational effort. Given Kaggle’s T4 GPU (16GB VRAM) constraints, each shift likely consumed 6–8 hours, totaling 30–40 GPU hours—nearly exhausting Kaggle’s weekly 30-hour free tier. For research, this approach highlights the trade-off between free-tier feasibility and training efficiency, suggesting that full-scale EEND-EDA training demands cloud GPUs (A100/V100) or distributed training for timely completion.

### 4.3.2 Training with EEND-DiaPer

The EEND-DiaPer model, trained over 4 shifts with varying epochs (2-speaker: 50, 3-speaker: 50, 4-speaker: 45, mixed: 30), posed even greater computational challenges. The extended epoch counts (totaling 175 epochs) drastically increased GPU time, likely requiring 45–50 hours on Kaggle’s T4 GPU—far exceeding free-tier limits and necessitating multiple weeks of quota management. This cost-intensive process underscores the scalability limitations of free-tier GPUs for complex diarization tasks.

## 4.4 Diarization Error Rate (DER)

The Diarization Error Rate (DER) is a metric used to evaluate the performance of speaker diarization systems. It quantifies the accuracy of the system by measuring the proportion of incorrectly identified speech segments compared to the total speech time. DER accounts for missed speech (when a speaker is not identified), false alarms (incorrectly identifying speech when there is none), and confusion errors (mislabeling speakers).

The formula for calculating DER is given by:

$$DER = \frac{Missed\ Speech + False\ Alarms + Confusion\ Errors}{Total\ Speech\ Time} \quad (4.1)$$

Where:

- **Missed Speech:** The total time where speech is present but not identified.
- **False Alarms:** The total time where speech is incorrectly identified.
- **Confusion Errors:** The total time of overlap where a different speaker is labeled than the actual speaker.
- **Total Speech Time:** The sum of all ground truth speech segments.

### 4.4.1 Different Models for DER

Speaker diarization is the task of determining “who spoke when” in an audio recording. In this work, we evaluate two architectures:

- **DiaPer:** Perceiver with cross-attention
- **EEND-EDA:** Encoder-Decoder architecture

Each architecture has four variants trained on different mixtures of speakers: 2-speaker, 3-speaker, 4-speaker, and M-speaker (mixed speaker). In total, we trained 8 models:

# 5. System design

## 5.1 EEND Model Pipeline Components

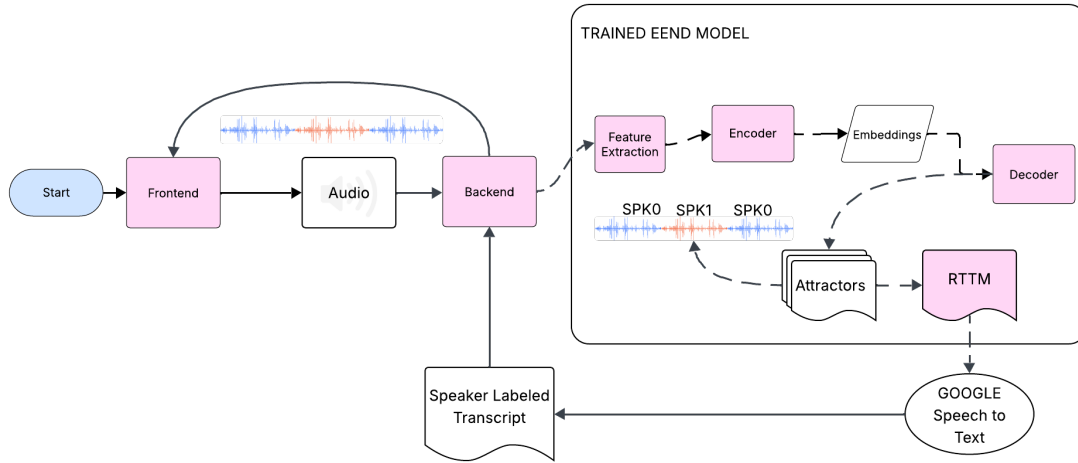


Figure 5.1: Encoder-Decoder Based EEND Architecture

### 5.1.1 Frontend (Audio Input)

The system processes raw audio files in standard formats (WAV, MP3) containing conversational speech with 2-4 speakers. The input audio typically consists of variable length segments ranging from 5 to 60 minutes, which are standardized to a 16kHz sampling rate for consistent processing. This stage handles the initial audio loading and format verification.

### 5.1.2 Feature Extraction

The audio signals are converted into spectral features using Mel spectrogram computation with 80 frequency channels. The analysis uses 25 millisecond windows with 10 millisecond overlap between frames, followed by log compression to improve the dynamic range representation. These features capture the essential spectral characteristics needed for speaker differentiation.

### 5.1.3 Encoder

A Transformer-based neural network with 12 layers processes the spectral features into meaningful speaker embeddings. The architecture employs multi-head attention mechanisms to

effectively model relationships across different time frames while maintaining the temporal structure of the conversation. This stage produces compact representations that encode speaker-specific characteristics.

#### **5.1.4 Attractor Calculation**

The system automatically estimates the number of speakers and creates unique profile representations (attractors) for each speaker present in the conversation. This component is particularly important for handling overlapping speech regions, where multiple speakers may be talking simultaneously. The attractors serve as reference points for distinguishing between different voices.

#### **5.1.5 Decoder & Speaker Labeling**

This stage matches speech segments to the calculated attractors and assigns speaker identity labels (SPK0, SPK1, etc.) to each segment. The process includes confidence scoring for each assignment, allowing for probabilistic interpretation of the results. The output provides a continuous speaker timeline throughout the conversation.

#### **5.1.6 RTTM Output**

The system generates standard Rich Transcription Time Marked (RTTM) format output that precisely timestamps speaker turns. A typical output line appears as: `SPEAKER meeting1 1 10.50 2.30 <NA> <NA> SPK0`, indicating the speaker ID, start time, and duration for each speech segment. This format is widely compatible with speech processing tools.

#### **5.1.7 Speech-to-Text Integration**

For applications requiring transcription, the system optionally processes speaker-separated audio through Google’s Speech-to-Text API. This integration produces time-aligned transcripts with speaker attribution, creating a complete record that shows who spoke what and when. The output combines diarization results with textual content for comprehensive conversation analysis.

# 6. Results and Output

## 6.1 Training and Hyperparameter Configuration

Speaker-diarization models were trained in four sequential phases: 2-speaker, 3-speaker, 4-speaker, then mixed-speaker (Mspk). Each phase has its own training and validation splits. Two architectures were trained in parallel:

- **DiaPer** – Attractor Perceiver with cross-attention

Table 6.1: DiaPer Training Parameters

| Option               | Value              |
|----------------------|--------------------|
| model_type           | AttractorPerceiver |
| feature_dim          | 40                 |
| frame_size           | 400                |
| train_batchsize      | 32                 |
| validation_batchsize | 128                |
| optimizer            | adam               |
| lr                   | 1e-5               |

- **EEND-EDA** – Transformer Encoder–Decoder Attractor

Table 6.2: EEND-EDA Training Parameters

| Option               | Value          |
|----------------------|----------------|
| model_type           | TransformerEDA |
| feature_dim          | 40             |
| frame_size           | 400            |
| train_batchsize      | 32             |
| validation_batchsize | 128            |
| optimizer            | adam           |
| lr                   | 1e-5           |

## 6.2 Dataset Composition

Each phase dataset is summarized below:



Table 6.3: Training and Validation Sets per Phase

| Phase     | Train Audio Count | Train Length (hr) | Validation Audio Count | Val Length (hr) |
|-----------|-------------------|-------------------|------------------------|-----------------|
| 2-speaker | 10000             | 145.1             | 3000                   | 37.67           |
| 3-speaker | 10000             | 161.37            | 3000                   | 48.49           |
| 4-speaker | 10000             | 215.90            | 3000                   | 64.53           |
| M-speaker | 10000             | 171.12            | 3000                   | 59.6            |

## 6.2.1 Training and Validation Curves

### DiaPer Training Curves for 2 Speakers

The DiaPer model was trained for a fixed 2-speaker diarization task using 10,000 training and 3,000 validation audio samples. Throughout training, both activation loss and DER consistently decreased, indicating effective learning and improved diarization performance. A small gap between training and validation DER suggests minor overfitting but overall good generalization.

**Epochs: 50**

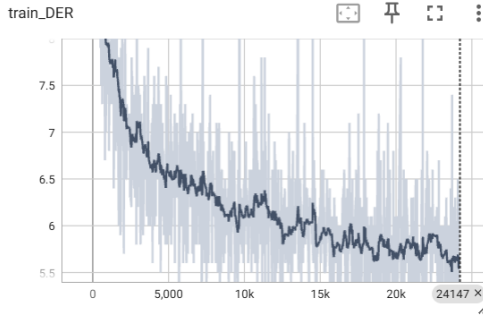


Figure 6.1: Train DER- 2spk

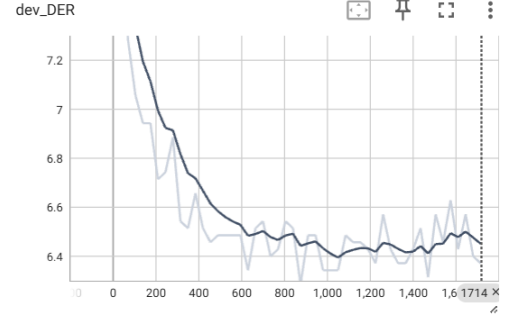


Figure 6.2: Valid DER - 2spk

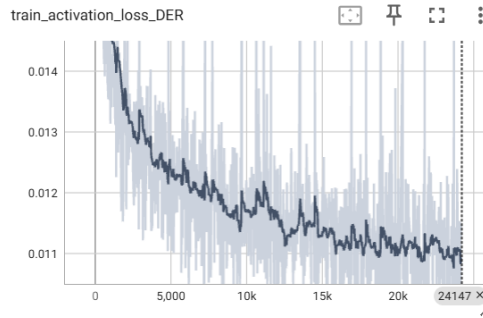


Figure 6.3: Train activation loss - 2spk

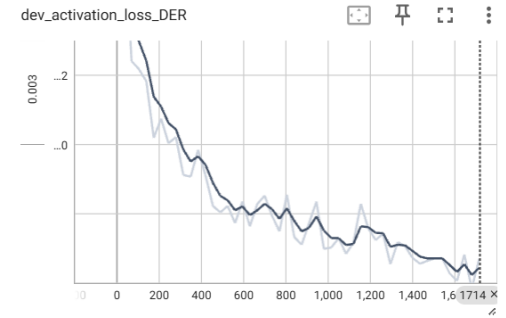


Figure 6.4: Valid activation loss - 2spk

## DiaPer Training Curves for 3 Speakers

The DiaPer model was trained for a fixed 3-speaker diarization task using 10,000 training and 3,000 validation audio samples. Throughout training, both activation loss and DER consistently decreased, indicating effective learning and improved diarization performance. While the validation DER was slightly higher, the gap remained small, indicating good learning despite increased speaker complexity.

**Epochs: 50**

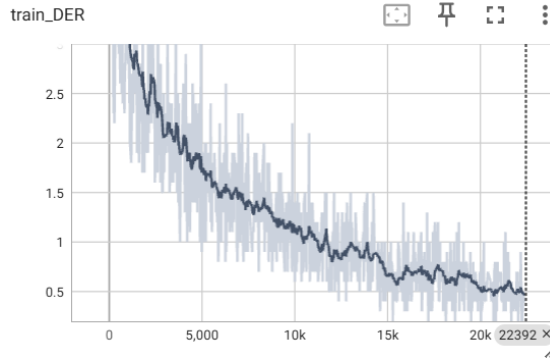


Figure 6.5: Train DER- 3spk

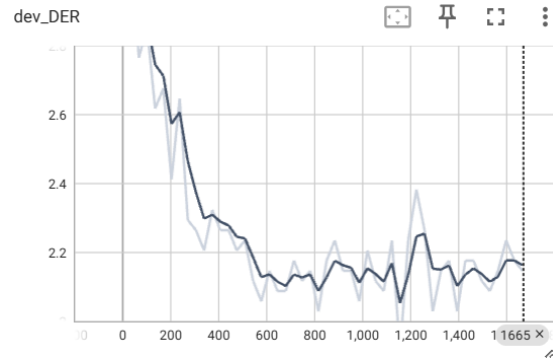


Figure 6.6: Valid DER - 3spk

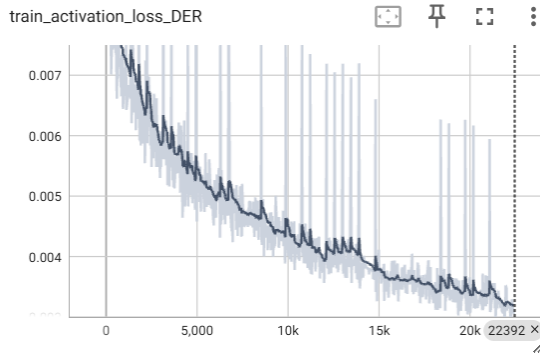


Figure 6.7: Train activation loss - 3spk

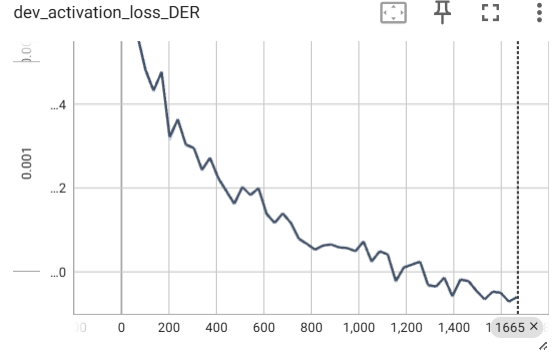


Figure 6.8: Valid activation loss - 3spk

## DiaPer Training Curves for 4 Speakers

Trained with 10,000 4-speaker mixtures and evaluated on 3,000 validation samples, the model showed a clear downward trend in both DER and loss. The validation metrics tracked the training ones well, suggesting effective handling of higher speaker counts.

**Epochs: 45**

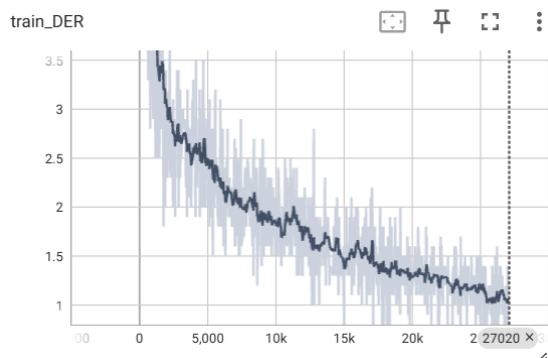


Figure 6.9: Train DER- 4spk

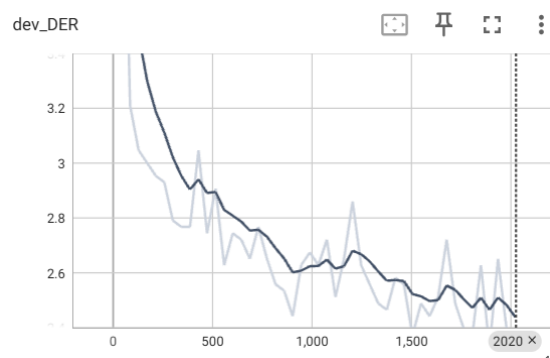


Figure 6.10: Valid DER - 4spk

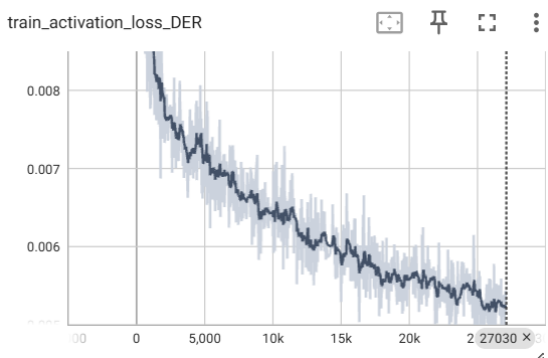


Figure 6.11: Train activation loss - 4spk

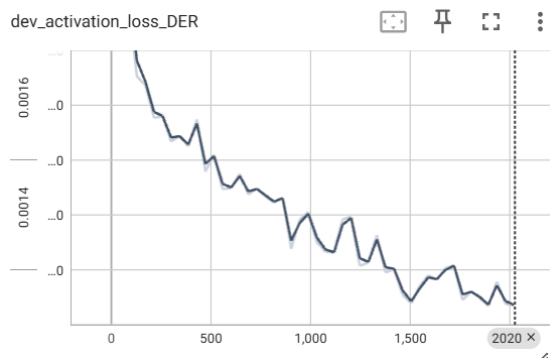


Figure 6.12: Valid activation loss - 4spk

## DiaPer Training Curves for M Speakers

Trained on 10,000 and validated on 3,000 audio clips with a mix of speaker counts, the model maintained a consistent decline in DER and loss. The close alignment of validation and training curves reflected strong generalization to variable speaker conditions.

**Epochs: 30**

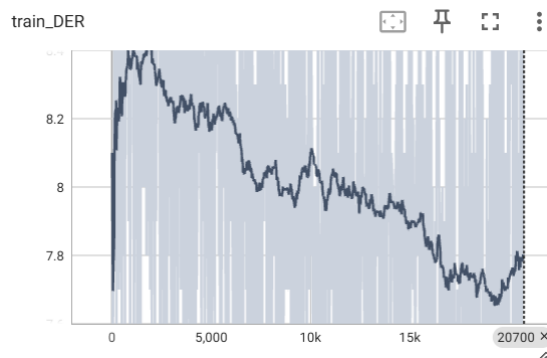


Figure 6.13: Train DER- Mspk

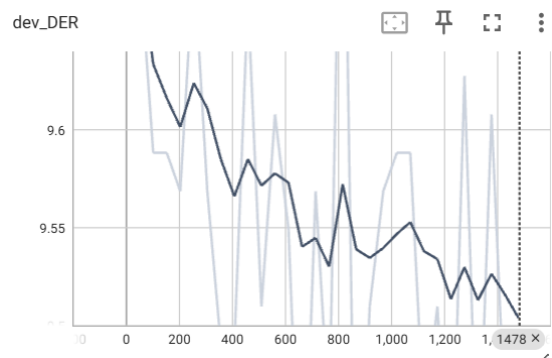


Figure 6.14: Valid DER - Mspk

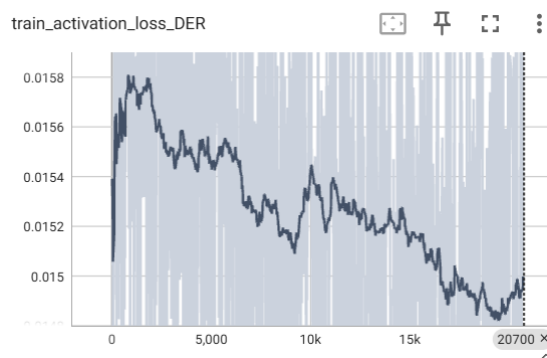


Figure 6.15: Train activation loss - Mspk

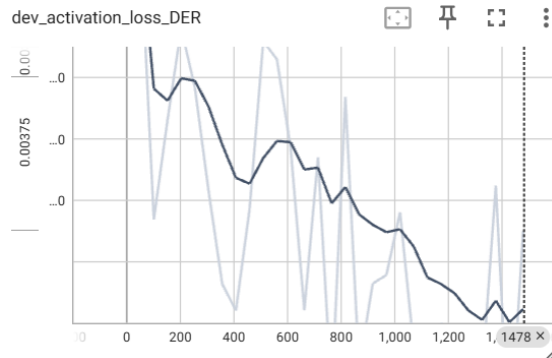


Figure 6.16: Valid activation loss - Mspk

## EEND-EDA Training Curves for 2 Speakers

Trained with 10,000 audios and validated on 3,000 clips (2 speakers each), the model shows consistent reduction in train/valid loss and DER\_miss. DER\_FA also converges steadily, indicating effective learning in the 2-speaker setup.

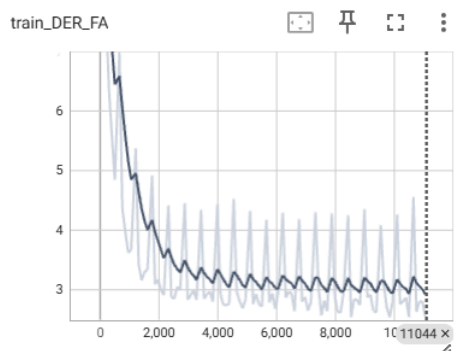


Figure 6.17: Train DER FA 2spk

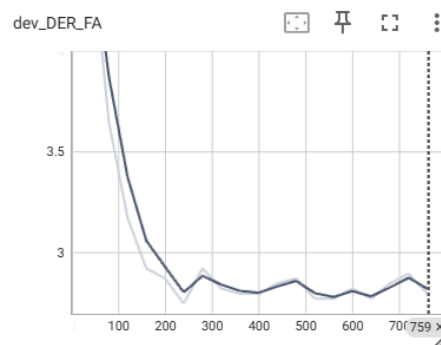


Figure 6.18: Valid DER FA 2spk

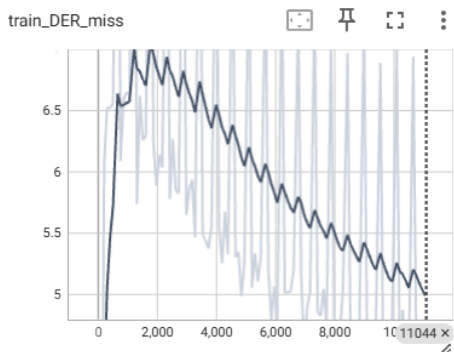


Figure 6.19: Train DER miss 2spk

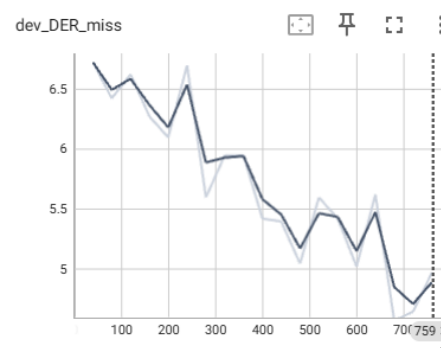


Figure 6.20: Valid DER miss 2spk

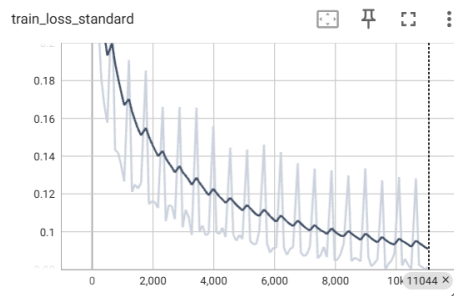


Figure 6.21: Train loss 2spk

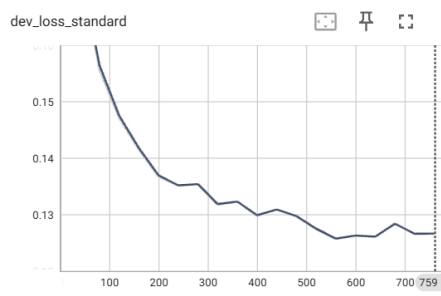


Figure 6.22: Valid loss 2spk

## EEND-EDA Training Curves for 3 Speakers

Using the same dataset (10k train, 3k valid), the model learns well in terms of loss and DER\_miss. However, DER\_FA shows slight fluctuations with a sudden rise after a certain point, suggesting early signs of instability in handling 3 speakers.

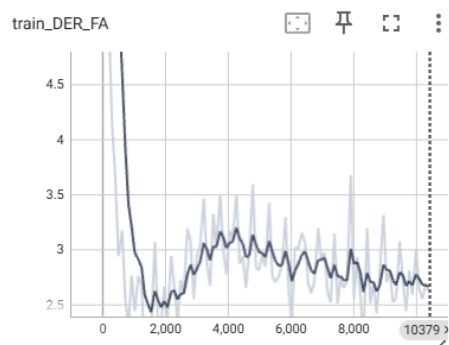


Figure 6.23: Train DER FA 3spk

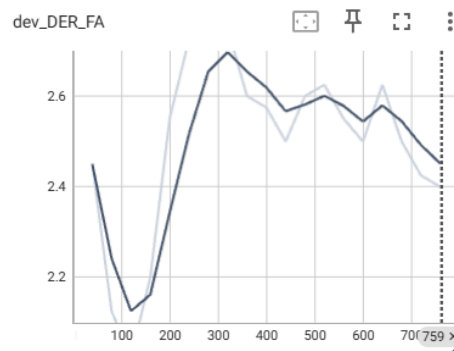


Figure 6.24: Valid DER FA 3spk

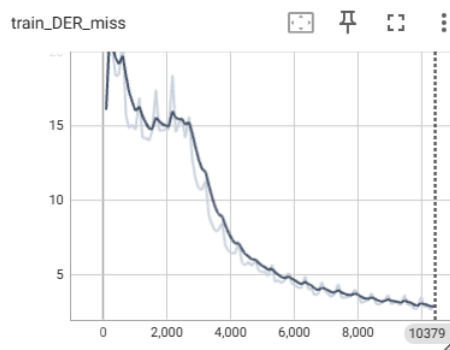


Figure 6.25: Train DER miss 3spk

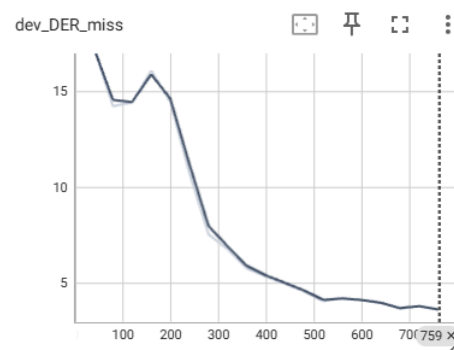


Figure 6.26: Valid DER miss 3spk

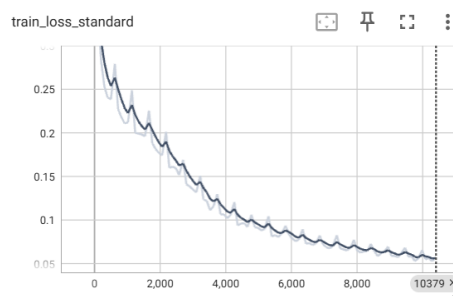


Figure 6.27: Train loss 3spk

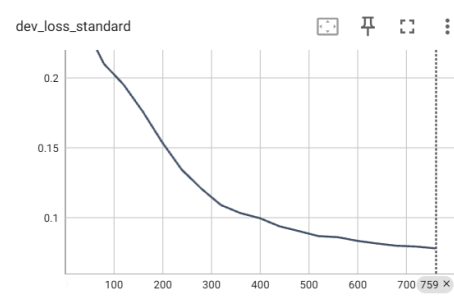


Figure 6.28: Valid loss 3spk

## EEND-EDA Training Curves for 4 Speakers

With 4-speaker audio samples (same data size), loss and DER\_miss continue to drop smoothly, but DER\_FA consistently rises with each epoch. This implies increasing false alarms as speaker count grows, highlighting a limitation of the model.

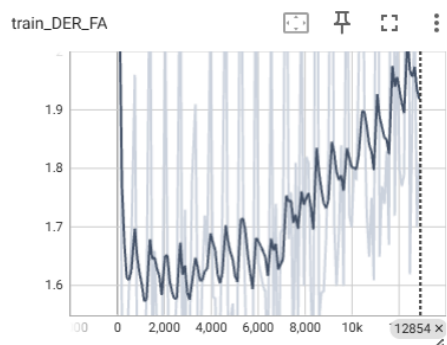


Figure 6.29: Train DER FA 4spk

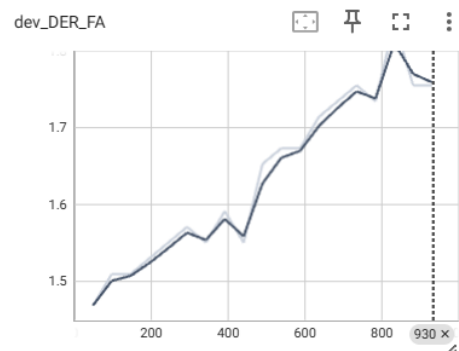


Figure 6.30: Valid DER FA 4spk

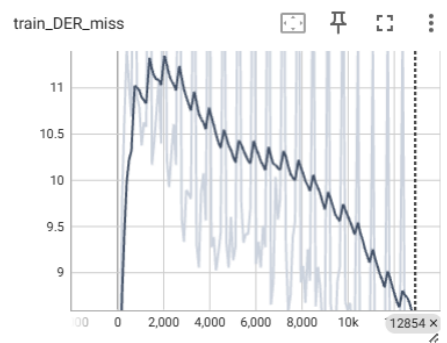


Figure 6.31: Train DER miss 4spk

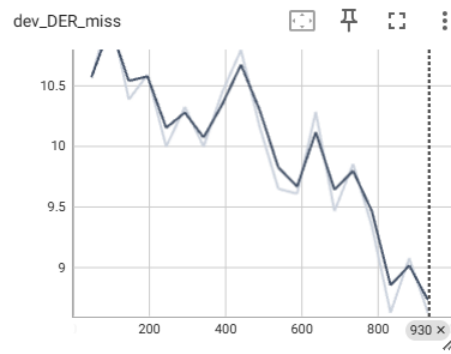


Figure 6.32: Valid DER miss 4spk

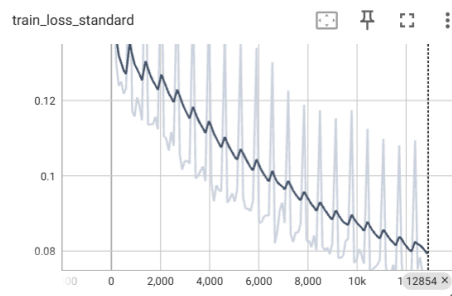


Figure 6.33: Train loss 4spk

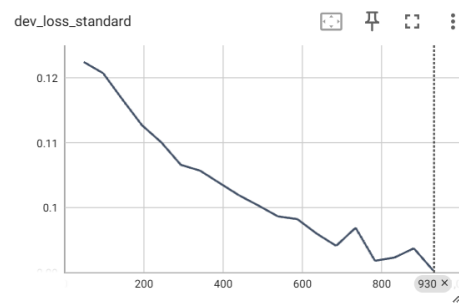


Figure 6.34: Valid loss 4spk

## EEND-EDA Training Curves for Mixed Speakers

Trained and validated on mixed-speaker audios (10k train, 3k valid), the model still shows declining loss and DER\_miss. However, DER\_FA increases gradually throughout training, confirming that EEND-EDA struggles to manage variable speaker scenarios.

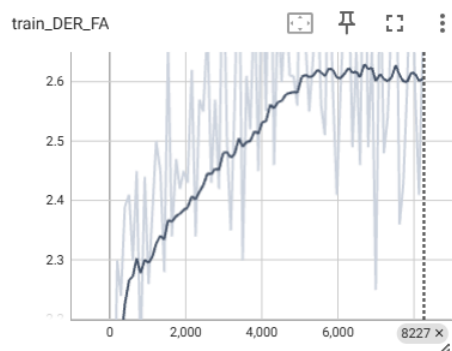


Figure 6.35: Train DER FA Mspk

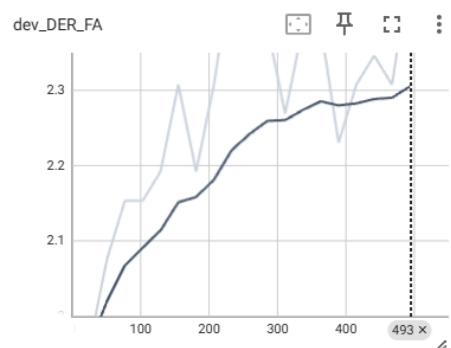


Figure 6.36: Valid DER FA Mspk

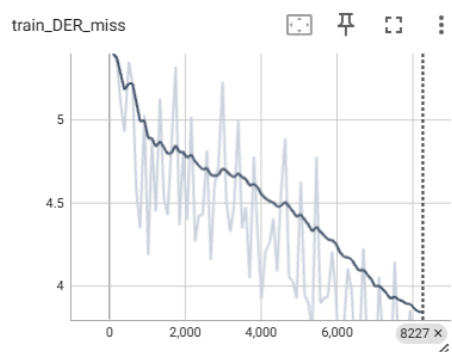


Figure 6.37: Train DER miss Mspk

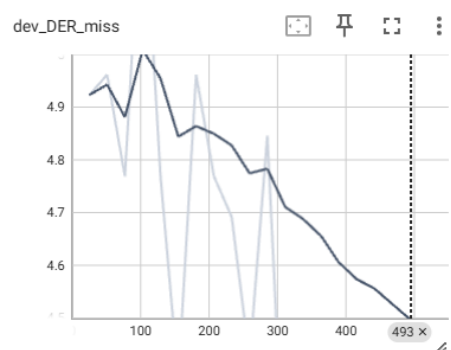


Figure 6.38: Valid DER miss Mspk

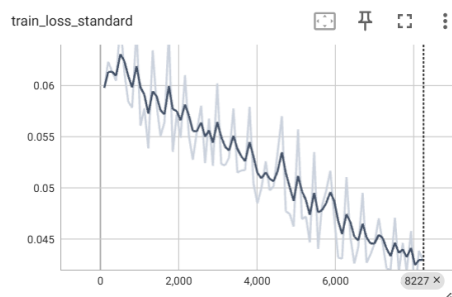


Figure 6.39: Train loss Mspk

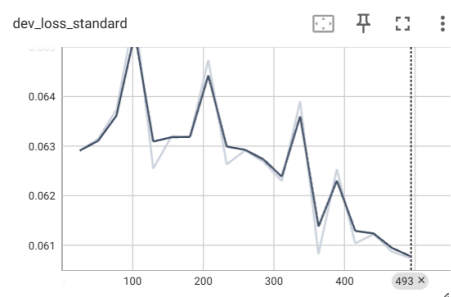


Figure 6.40: Valid loss Mspk



## 6.3 Testing & Output

Testing was performed using the Diarization Error Rate (DER) metric to evaluate the accuracy of the speaker diarization. However, for Nepali audio, we manually tested audio clips collected from YouTube and other sources. The output of the testing phase was an RTTM file, which logs the results of the diarization. Below is an example of the RTTM output:

```
SPEAKER nepali2 1 0.0 28.3 <NA> <NA> spk2 <NA> <NA>
SPEAKER nepali2 1 29.7 18.2 <NA> <NA> spk1 <NA> <NA>
SPEAKER nepali2 1 50.6 4.3 <NA> <NA> spk0 <NA> <NA>
SPEAKER nepali2 1 55.0 4.1 <NA> <NA> spk1 <NA> <NA>
```

This output logs speaker segments with their respective timings, allowing us to evaluate the model’s performance in correctly identifying and separating speakers.

### 6.3.1 Testing Without Noise Cancellation

During the initial tests, when noise cancellation was not applied, we encountered an issue where the model mistakenly diarized silent segments as a separate speaker. This resulted in both active speakers being assigned to the same speaker ID, while the silent portions were incorrectly assigned to the other speaker ID.

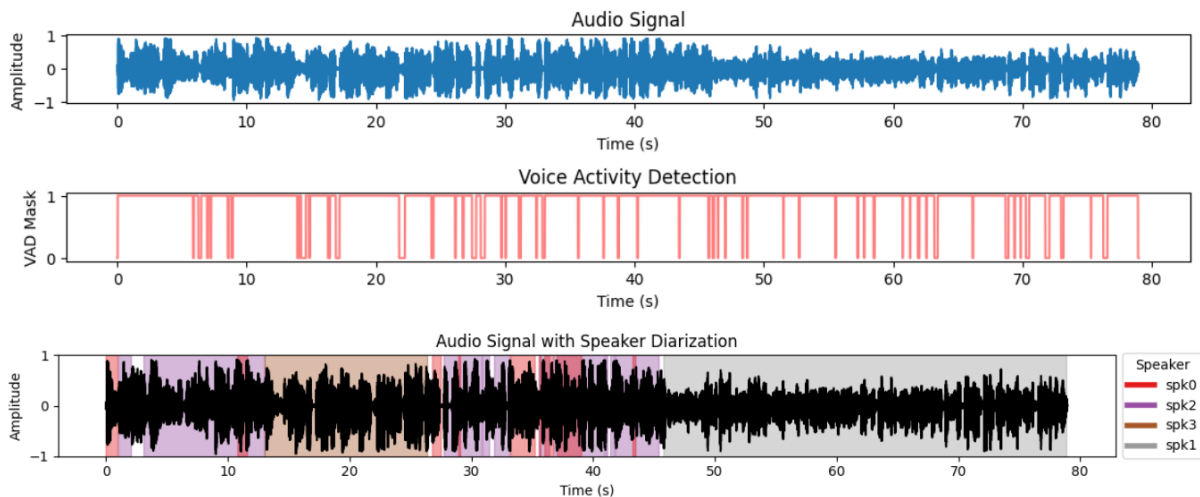


Figure 6.41: Incorrect diarization without noise cancellation.

### 6.3.2 Testing With Noise Cancellation

The issue was resolved by applying noise cancellation, which significantly improved diarization accuracy. By removing the silent sections from the audio, the model was better able to

distinguish between the two speakers, leading to a significant improvement in the Diarization Error Rate (DER) score.

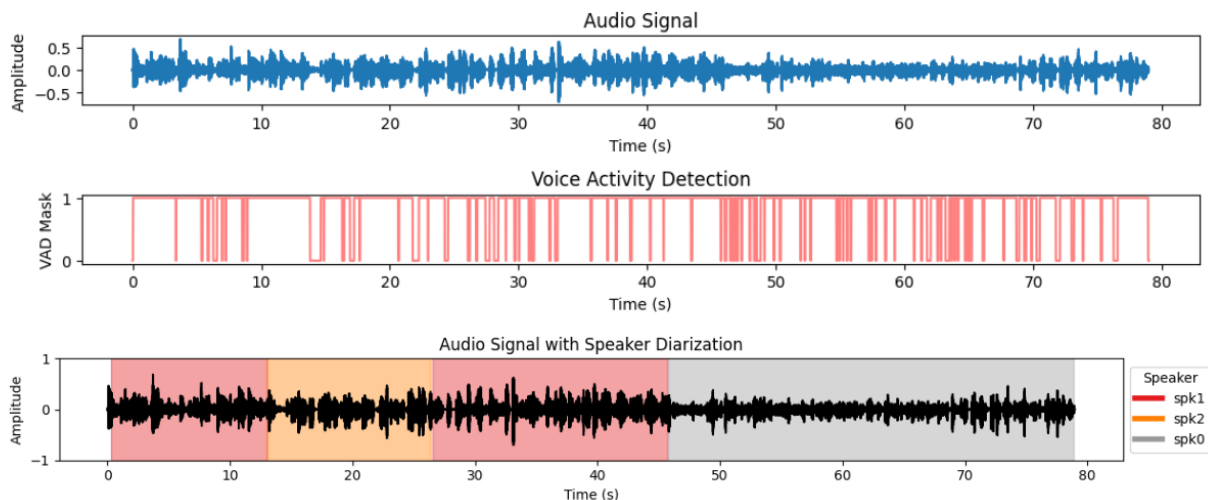


Figure 6.42: Correct diarization after applying noise cancellation.

## 6.4 Testing Datasets and DER Evaluation

DiaPer-2spk, DiaPer-3spk, DiaPer-4spk, DiaPer-Mspk,  
EEND-2spk, EEND-3spk, EEND-4spk, and EEND-Mspk.

### 6.4.1 Testing Datasets

We evaluated each model on four sources:

- **LibriSpeech** (2/3/4/Mspk)
- **NeHi** (2/3/4/Mspk)
- **VoxCeleb** (2/3/4/Mspk)

Table 6.4: Testing-set Composition (audio count &amp; total duration)

| <b>Dataset</b>   | <b>Total Audio No.</b> | <b>Total Length (hr)</b> |
|------------------|------------------------|--------------------------|
| LibriSpeech-2spk | 1000                   | 11.87                    |
| LibriSpeech-3spk | 1000                   | 17.95                    |
| LibriSpeech-4spk | 1000                   | 24.21                    |
| LibriSpeech-Mspk | 1000                   | 18.52                    |
| NeHi-2spk        | 100                    | 0.76                     |
| NeHi-3spk        | 100                    | 1.19                     |
| NeHi-4spk        | 100                    | 1.64                     |
| NeHi-Mspk        | 100                    | 1.21                     |
| VoxCeleb-2spk    | 2000                   | 19.66                    |
| VoxCeleb-3spk    | 2000                   | 29.91                    |
| VoxCeleb-4spk    | 2000                   | 39.84                    |
| VoxCeleb-Mspk    | 2000                   | 29.46                    |

### 6.4.2 DER Evaluation

Diarization Error Rate (DER) is computed on each testing set for all eight models. Below, “DiaPer- $k$ spk” and “EEND- $k$ spk” are the two architectures’ variants.

#### LibriSpeech

Table 6.5: DER (%) on LibriSpeech test segments

| <b>Dataset/model</b> | DiaPer-2spk | DiaPer-3spk | DiaPer-4spk | DiaPer-Mspk | EEND-2spk | EEND-3spk | EEND-4spk | EEND-Mspk |
|----------------------|-------------|-------------|-------------|-------------|-----------|-----------|-----------|-----------|
| LibriSpeech-2spk     | 1.55        | 8.25        | 7.53        | 5.22        | 1.43      | 5.21      | 6.21      | 4.89      |
| LibriSpeech-3spk     | 23.32       | 2.99        | 5.59        | 5.29        | 37.16     | 8.31      | 10.68     | 8.95      |
| LibriSpeech-4spk     | 37.17       | 20.24       | 5.56        | 7.71        | 56.53     | 25.50     | 18.73     | 18.41     |
| LibriSpeech-Mspk     | 21.45       | 9.67        | 5.66        | 5.70        | 43.63     | 18.15     | 11.48     | 10.50     |

#### NeHi

Table 6.6: DER (%) on NeHi test segments

| <b>Dataset/model</b> | DiaPer-2spk | DiaPer-3spk | DiaPer-4spk | DiaPer-Mspk | EEND-2spk | EEND-3spk | EEND-4spk | EEND-Mspk |
|----------------------|-------------|-------------|-------------|-------------|-----------|-----------|-----------|-----------|
| NeHi-2spk            | 3.28        | 5.31        | 2.99        | 0.88        | 1.50      | 4.00      | 4.73      | 3.19      |
| NeHi-3spk            | 24.44       | 2.02        | 2.51        | 4.16        | 44.28     | 9.68      | 10.80     | 8.96      |
| NeHi-4spk            | 35.34       | 18.90       | 4.05        | 8.20        | 68.14     | 25.94     | 16.17     | 15.44     |
| NeHi-Mspk            | 21.73       | 9.02        | 5.83        | 4.76        | 55.50     | 19.07     | 10.64     | 11.19     |

## VoxCeleb

Table 6.7: DER (%) on VoxCeleb test segments

| Dataset/model | DiaPer-2spk | DiaPer-3spk | DiaPer-4spk | DiaPer-Mspk | EEND-2spk | EEND-3spk | EEND-4spk | EEND-Mspk |
|---------------|-------------|-------------|-------------|-------------|-----------|-----------|-----------|-----------|
| VoxCeleb-2spk | 1.14        | 5.06        | 5.78        | 2.44        | 1.82      | 4.41      | 5.61      | 3.09      |
| VoxCeleb-3spk | 21.20       | 1.11        | 1.89        | 1.73        | 42.71     | 7.06      | 8.41      | 6.95      |
| VoxCeleb-4spk | 33.93       | 17.08       | 1.94        | 3.17        | 65.8      | 23.33     | 17.40     | 16.91     |
| VoxCeleb-Mspk | 18.48       | 7.04        | 2.87        | 2.60        | 53.14     | 18.40     | 10.01     | 8.99      |

## 6.5 Web Interface

To effectively visualize speaker diarization, we developed a web-based application that processes audio recordings and assigns speaker labels dynamically. The system identifies distinct speakers in a given conversation and labels them as **spk0**, **spk1**, **spk2**, etc. The web interface displays the transcriptions along with their respective time intervals, providing a clear understanding of speaker transitions.

This implementation is particularly useful for analyzing Nepali conversational data, where multiple speakers interact in a dialogue. Given the lack of large-scale multispeaker Nepali datasets, our system helps bridge this gap by offering structured diarization output for further analysis. The web application also enhances accessibility by allowing users to interactively explore the diarized segments, improving the clarity and usability of speech processing results.



Figure 6.43: Web App demonstration of Speaker Diarization.

# 7. Limitations

## 7.1 Speaker Independent Accuracy and Robustness

One of the biggest challenges in speaker diarization is maintaining accuracy across different numbers of speakers. Most models, including the one used in this project, are trained on a specific dataset with a fixed number of speakers (e.g., four). This creates a significant limitation: when the number of actual participants in an audio recording is different—either more or fewer—the performance of the diarization model tends to degrade.

### 7.1.1 Causes of Accuracy Drop

The reason behind this drop in accuracy lies in how machine learning models generalize patterns. During training, the model learns to recognize voice characteristics and speech patterns within a controlled environment, where it expects a fixed number of unique speakers. However, in real-world scenarios, conversations are dynamic—some recordings may have only two or three speakers, while others might have five or more. When the model encounters a different number of speakers than it was trained on, it struggles to adapt due to the following reasons:

- **Over-Splitting or Merging Speakers:** If there are fewer speakers than expected, the model may mistakenly assign multiple speaker labels to the same person, resulting in a single individual appearing to be “split” into different speakers. Conversely, when there are more speakers than the model is trained to handle, it might merge two distinct voices under the same label, failing to distinguish between different participants.
- **Speaker Embeddings Become Less Reliable:** The model relies on embeddings which is mathematical representations of each speaker’s voice—to differentiate between speakers. However, when faced with an unfamiliar speaker count, these embeddings may overlap, leading to errors in identifying who is speaking.
- **Short Utterances and Speaker Switching:** In natural conversations, people often interrupt or speak in short bursts. If the model is not trained on highly dynamic dialogues with a varying number of speakers, it may struggle to assign the correct speaker labels in these fast-paced interactions.

## 7.2 Training Data Limitations

One of the major challenges in developing an accurate speaker diarization system is the availability of suitable training data. While there is an abundance of single-speaker audio datasets, there is a noticeable scarcity of high-quality, multispeaker conversational datasets. Speaker diarization models require audio recordings with multiple speakers interacting naturally, but most publicly available datasets are designed for isolated speech, making it difficult for models to generalize to real-world conversations.

Additionally, there is a significant gap in datasets for non-English languages, particularly Hindi and other Eastern languages. Most existing speaker diarization models are trained on English-centric datasets, which limits their effectiveness in multilingual environments. Nepali, for example, has diverse accents and dialectal variations, which are underrepresented in available datasets.

To improve diarization performance, there is a need for larger, high-quality multispeaker datasets that cover diverse conversational scenarios and languages.

## 7.3 Speaker Labeling Limitations

Speaker labeling is a crucial step in diarization, where the system assigns unique identifiers to different speakers in an audio recording. Our current system faces a significant limitation it can effectively segment and differentiate speakers but cannot automatically infer their real names from the given transcription.

- **Anonymous Speaker Labels:** The system currently assigns generic labels such as `Speaker 1`, `Speaker 2`, etc., instead of actual speaker names. This means that while different speakers are correctly segmented, the output lacks personalization, making it difficult to map diarized segments to real individuals without manual intervention.
- **Lack of Speaker Identity Learning:** Even when transcriptions contain names or identifiable context, the model does not leverage this information to associate a speaker’s voice with their identity. Unlike advanced speaker recognition systems that use pretrained identity embeddings, our system operates purely on voice segmentation and cannot perform speaker identification.

Addressing this limitation would require integrating speaker recognition models that can match speaker embeddings with known identities based on labeled training data or transcription context. Future improvements could involve fine-tuning diarization models with identity-aware learning, leveraging metadata, or incorporating speaker verification techniques.

## 8. Future Enhancements

As speaker diarization technology continues to evolve, several enhancements can be introduced to improve accuracy, usability, and real-world applications. This chapter outlines three major areas of improvement: Large Language Models (LLMs) for speaker labeling, visualizing speaker diarization with animations and Nepali text, and achieving real-time diarization.

### 8.1 LLM-Powered Speaker Labeling

Currently, our system effectively separates and diarizes speakers but does not assign actual names to them based on the transcript. Large Language Models (LLMs) can be integrated to analyze transcriptions and infer speaker identities in a meaningful way.

- **Context-Based Speaker Identification:** LLMs can analyze the flow of conversation and infer who is speaking based on linguistic patterns, past references, or role-based dialogues (e.g., a teacher and student discussion).

Integrating LLMs for speaker labeling would enhance diarization outputs, making them more interpretable and useful for real-world applications such as meetings, interviews, and media transcription.

### 8.2 Visualization with Animations and Nepali Text

Currently, diarization results are primarily stored in RTTM format, making them difficult for general users to interpret. A more intuitive approach would involve animated visualizations, where speakers are dynamically represented, and Nepali text is displayed alongside their speech.

- **Nepali Text Rendering:** Instead of only providing English transcriptions, the system could display synchronized Nepali subtitles, making the diarization system accessible to a wider audience.
- **Timeline-Based Interaction:** Users could scroll through the timeline of a conversation and see the animated transitions between speakers, improving readability and engagement.

By adding these visualization features, the diarization results would become more interactive, moving beyond simple text-based logs to a more user-friendly experience.

## 8.3 Real-Time Speaker Diarization

Currently, the diarization process works in an offline manner, where audio is first recorded, then processed for speaker segmentation. However, real-time speaker diarization would significantly improve applications such as live meetings, broadcast media, and virtual assistants.



# References

- [1] Yusuke Fujita et al. End-to-end neural speaker diarization with self-attention. *arXiv preprint arXiv:1909.06247*, 2019.
- [2] Shota Horiguchi et al. End-to-end speaker diarization for an unknown number of speakers with encoder-decoder based attractors. *arXiv preprint arXiv:2005.09921*, 2020.
- [3] Federico Landini, Mireia Diez, Themis Stafylakis, and Lukáš Burget. Diaper: End-to-end neural diarization with perceiver-based attractors. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 32:3450–3465, 2024.
- [4] Vassil Panayotov, Daniel Povey, Guoguo Chen, and Sanjeev Khudanpur. Librispeech asr corpus. OpenSLR, Identifier SLR12, 2021.
- [5] Arsha Nagrani, Joon Son Chung, and Andrew Senior. Voxceleb: A large-scale speaker identification dataset. *arXiv preprint arXiv:1706.08612*, 2017.
- [6] Keshan Sodimana, Knot Pipatsrisawat, Linne Ha, Martin Jansche, Oddur Kjartansson, Pasindu De Silva, and Supheakmongkol Sarin. A Step-by-Step Process for Building TTS Voices Using Open Source Data and Framework for Bangla, Javanese, Khmer, Nepali, Sinhala, and Sundanese. In *Proc. The 6th Intl. Workshop on Spoken Language Technologies for Under-Resourced Languages (SLTU)*, pages 66–70, Gurugram, India, August 2018.
- [7] Shivam Shukla. Speech dataset in hindi language, 2020.
- [8] Federico Landini, Alicia Lozano-Diez, Mireia Diez, and Lukáš Burget. From Simulated Mixtures to Simulated Conversations as Training Data for End-to-End Neural Diarization. In *Proc. Interspeech 2022*, pages 5095–5099, 2022.
- [9] Federico Landini, Mireia Diez, Alicia Lozano-Diez, and Lukáš Burget. Multi-speaker and wide-band simulated conversations as training data for end-to-end neural diarization. In *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 1–5. IEEE, 2023.